CIS 6930 / 4930 · GENERATIVE AI · UNIVERSITY OF SOUTH FLORIDA

Course Project — Guidelines, Topics & Rubric

Prof. Nima Karimian · Fall 2026 · Submit via Canvas · Collaboration Policy: Individual work

This document defines the semester project: how to choose a topic, what your proposal must contain, how the work is graded, and when each piece is due. The project is worth **100 points**. **65 of those points are earned live** — in your presentation and the unscripted Q&A that follows it — because that is where understanding, as opposed to output, can actually be measured.

Part 1 covers scope, compute, and the proposal. **Part 2** lists suggested topics with the papers behind them. **Part 3** is the timeline and presentation slots. **Part 4** is the rubric. AI coding assistants are permitted and must be declared (§4.5). Everyone works individually.

PART 1 Scope, Compute & Approval

Proposal: 5 pts

1.1 What Counts as a Project

A project is in scope if it has a question, an implementation, and an evaluation. If any one of the three is missing, it will not be approved.

Every project must include all of the following

1. A stated research question or hypothesis, written in **one sentence**.
2. A **primary paper** your work is anchored to, named in the proposal, plus two to four supporting references.
3. Code you wrote or modified — a model you train, fine-tune, adapt, attack, or defend. Calling a hosted API and collecting the outputs is not an implementation.
4. A quantitative evaluation on at least one dataset, against at least one **baseline** — the original method, a simpler method, or an ablated version of your own.
5. At least one controlled comparison or ablation that isolates the effect of the thing you changed.
6. An honest failure analysis: what did not work, and your explanation of why.

What will not be approved

- Prompt engineering with no training, fine-tuning, or quantitative evaluation.
- A UI or wrapper around an existing API or hosted model.

- Running a public repository's README end to end and reporting its published numbers.
- A literature survey or slide summary with no implementation.
- Work submitted for credit in another course, or research that was already complete before this semester. Ongoing research you extend *for this course* is fine — say so in the proposal and state precisely what is new.

Negative results are acceptable. A well-designed experiment showing that your idea did not beat the baseline, with a credible analysis of why, can earn full marks. A poorly designed experiment that happens to produce a nice number cannot.

1.2 Compute: GAIVI and Your Compute Plan

Compute is not a limiting factor in this course. Scope your project by what you can think through in eight weeks, not by what fits on a free Colab session.

GAIVI, the USF GPU cluster, is available to you for this project. Request an account early — before the proposal deadline, not after — and run your first trivial job the week you get access, so that you discover the environment problems while they are still cheap. Jobs are scheduled; you submit batch jobs and wait in a shared queue rather than getting an interactive GPU on demand.

Google Colab, Kaggle, a lab machine, or your own GPU are all equally acceptable. Use whichever gets the work done.

You still owe a compute plan

Because the cluster is shared, **queue time is part of your schedule**, and it is the single most common reason a project arrives at the presentation unfinished. Your proposal must therefore state:

- Where you will run — GAIVI, Colab, local, or a mix — and whether you already have access.
- Your estimated GPU-hours, with the arithmetic shown: `(steps × sec/step) ÷ 3600 × runs`.
- A **reduced-scale fallback** that runs end to end on a single GPU in under two hours — smaller subset, lower resolution, fewer steps, LoRA instead of full fine-tuning. You will use it for debugging, and you will be glad of it in week 7.

Having a large cluster raises the ceiling on what you can attempt. It does not change the deadline. *"The queue was long"* and *"my job kept getting preempted"* are not accepted explanations for missing results — they are the predictable conditions you are being asked to plan around.

Housekeeping

- Do not leave idle interactive sessions holding GPUs. Other students in this class are waiting behind you.
- Checkpoint often enough that a preempted or timed-out job costs you minutes, not days.
- Log every run — config, seed, git commit, and result. You will be asked in the Q&A which run produced a number on your slide.
- Follow the cluster's own storage and acceptable-use rules. Do not stage datasets you have no right to distribute.

1.3 Choosing a Topic

Two paths, neither graded more favorably than the other.

Path A — propose your own. Bring a problem you care about: something from your research, a paper you want to test, a claim you think is wrong. More risk (it is easier to over-scope) and more upside (original questions are easier to defend in Q&A because they are yours).

Path B — pick from the suggested topics in Part 2. Each comes with its primary paper. Picking one is not a lesser choice; you still have to define your own question, baseline, and evaluation within it.

RULE	DETAIL
Approval is mandatory	No work counts until your proposal is approved (§1.5).
Stay in the subject area	Generative modeling, vision, multimodal, or the security and evaluation of these. If you are unsure whether something qualifies, ask before Oct 5, not after.
Name a primary paper	From **2021 or later** — CVPR, ICCV, ECCV, NeurIPS, ICLR, ICML, ACL, EMNLP, or a serious arXiv preprint. Older foundations (GANs, ResNet, ViT, DDPM) may be used as baselines.
No duplicate projects	Two students may work in the same area, but not on the same question with the same dataset and the same method. First come, first served by proposal timestamp.
Data must be public or yours	Established public datasets, or data you own and have the right to use. No scraping of personal data, no identifiable-person datasets without documented consent, no human-subjects data collection.
Declare the base repository	If a public implementation of your starting point exists, name it in the proposal whether or not you use it (§1.4).

A note on adversarial topics. Attack and defense projects are welcome and encouraged. Evaluate them on public benchmarks and on your own models. Do not attack a deployed commercial system, a production API, or anyone's account or infrastructure — including USF systems and GAIVI itself. A project that would require this will not be approved.

1.4 Building on Existing Work

You may start from a public repository. Most of you should. You are graded on the delta, not on the base.

Reimplementing a diffusion model from scratch in eight weeks is not a good use of your time, and standing on a working codebase is what real research does. The condition is simple: if a repository already does what you are demonstrating, you have submitted nothing.

COUNTS AS A CONTRIBUTION	WHAT IT LOOKS LIKE
New capability	A component, loss, conditioning path, attack, or defense the base repository does not implement

COUNTS AS A CONTRIBUTION	WHAT IT LOOKS LIKE
New domain or data	Porting the method to a dataset or modality it was not built for, and handling what breaks
New evaluation	A metric, probe set, or protocol that tests a claim the authors did not test
Skipped ablation	Removing or varying a component the authors asserted was necessary, and reporting what actually happens
Corrective result	A documented failure to reproduce, with the cause isolated and evidence for it
Efficiency	Matching reported results at materially lower compute, memory, or steps — measured, not asserted

What does not count

- Running the base repository and reporting its numbers.
- Changing hyperparameters with no question attached to the change.
- Swapping in a different pretrained checkpoint.
- Rewriting working code in another framework, or adding a UI, a notebook, or a Dockerfile.
- Adding a README, plots, or documentation — good practice, but not a contribution.

REQUIRED — CONTRIBUTIONS.MD, IN THE PROPOSAL AND IN THE REPORT

```
| Component      | Origin                                  | What I changed
|
| -------------- | --------------------------------------- | ---------------------------------------
----- |
| Training loop  | github.com/org/repo @ commit a1b2c3d    | Unmodified
|
| Sampler        | github.com/org/repo @ commit a1b2c3d    | Rewrote step schedule; added early
exit     |
| Probe dataset  | Mine                                    | Built from scratch, 420 labeled
examples    |
| Eval harness   | Mine, adapted from repo/eval.py         | New metric, new per-category breakdown
|
```

Pin the base commit, and let your own work appear as **commits over weeks**. A repository whose entire content arrives in a single commit on the due date is graded as unverifiable provenance and loses most of the 15 code points. Undeclared use of someone else's code is plagiarism and is handled under the integrity policy, not this section.

1.5 The Proposal

5 pts · due Oct 5

Two pages maximum excluding references, PDF, due Monday Oct 5 at 11:59 PM on Canvas.

No work you do before approval is guaranteed to count. If your topic is rejected you start over, and there is not much semester left in which to do that.

Required contents, in this order

1. **Header** — title, your name, and whether you are enrolled as **undergraduate (4930)** or **graduate (6930)**. This fixes which bar in §4.4 you are held to and cannot be changed later.
2. **Question** — one paragraph of context ending in a single sentence that begins *"This project asks whether…"* or *"This project measures…"*
3. **Primary paper and background** — the paper you are anchored to, plus two to four supporting references, cited properly. One sentence each on what they establish, then one sentence on the gap you address.
4. **Method** — what you will implement, at the detail of a block diagram in words. Name the model, the loss, and the training procedure.
5. **Data** — dataset, size, license, and the subset you will actually use. If you are building a probe or evaluation set, say how many examples and how they will be labeled.
6. **Evaluation plan** — metrics, baselines, and one sentence naming **what result would show your hypothesis is wrong**. A proposal with no falsifiable outcome is returned.
7. **Compute plan** — per §1.2, including the single-GPU fallback.
8. **Milestones** — a dated line per week from approval to your presentation slot.
9. **Risk and fallback** — in the form *"if X is not working by [date], I will fall back to Y."* Every project needs one.
10. **Contribution table** — required if building on existing code (§1.4).
11. **AI tool declaration** — one line (§4.5).

APPROVAL LOOP	WHEN
Proposal due	**Mon, Oct 5 · 11:59 PM**
Written feedback returned — approved, approved with conditions, or revise	**by Fri, Oct 9**
Revised proposal due, if you were asked to revise	**Mon, Oct 12 · 11:59 PM**

One revision carries **no grade penalty** — the 5 points are scored on the revised version. This is deliberate: a returned proposal means the project would have failed, and catching that on Oct 9 is worth far more to you than 5 points. Approval must be in hand by **Oct 12**; after that, topic changes require a documented technical blocker and an email *before* the change, not after.

The most common reasons a proposal is returned

- Too big — three training runs of a model you have never trained is not a semester project, however much cluster time exists.
- No baseline. "I will build X" is not a research question; "is X better than Y at Z, measured by M" is.
- No falsifiable outcome — written so that any result counts as success.
- The contribution over the base repository is unstated or cosmetic.
- No compute plan, no fallback, or arithmetic that is wrong by an order of magnitude.

If you are unsure whether your idea qualifies, bring it to office hours **before** Oct 5. That conversation costs ten minutes; a rejected proposal costs a week.

PART 2 Suggested Topics

21 options

Each entry is a *starting point*, not a finished project — you still narrow it to one question, pick a baseline, and decide what you measure. The papers listed are where to start reading, not a required reading list. Anything in these areas that is not on this list is Path A: propose it.

AREA A Diffusion & Generative Models

5 topics

- **1 · Guidance scale, fidelity, and diversity.** Sweep the classifier-free guidance scale on a small class-conditional model and measure both sides of the trade-off with FID and precision/recall, rather than FID alone.

  Read: Ho & Salimans, "Classifier-Free Diffusion Guidance," NeurIPS 2021 Workshop · Kynkäänniemi et al., "Improved Precision and Recall Metric for Assessing Generative Models," NeurIPS 2019

- **2 · How few sampling steps can you get away with?** Compare DDIM step counts against a distilled few-step sampler on the same model; report a quality-versus-wall-clock curve, not a single number.

  Read: Song, Meng & Ermon, "Denoising Diffusion Implicit Models," ICLR 2021 · Salimans & Ho, "Progressive Distillation for Fast Sampling of Diffusion Models," ICLR 2022 · Song et al., "Consistency Models," ICML 2023

- **3 · Personalization under a fixed budget.** DreamBooth vs. LoRA vs. Textual Inversion given exactly five reference images; measure identity preservation against prompt fidelity and show where each method breaks.

  Read: Ruiz et al., "DreamBooth," CVPR 2023 · Gal et al., "An Image is Worth One Word: Textual Inversion," ICLR 2023 · Hu et al., "LoRA," ICLR 2022

- **4 · Conditional control on a narrow domain.** Train a small ControlNet-style conditioning branch (edge or depth) on one domain and measure condition adherence with a metric you design and defend.

  Read: Zhang, Rao & Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," ICCV 2023

- **5 · Does the watermark survive the internet?** Test whether generated-image watermarks remain detectable after JPEG, crop, resize, and re-generation.

  Read: Fernandez et al., "The Stable Signature: Rooting Watermarks in Latent Diffusion Models," ICCV 2023 · Wen et al., "Tree-Rings Watermarks: Invisible Fingerprints for Diffusion Images," NeurIPS 2023

AREA B Transformers, Fine-Tuning & Post-Training

6 topics

- **6 · Where does the LoRA rank curve flatten?** Sweep rank from 1 to 64 on one model and one benchmark; plot task performance against trainable parameters and explain the shape of the curve.

  Read: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022

- **7 · What does quantization actually cost?** Full vs. 8-bit vs. 4-bit fine-tuning on accuracy, peak memory, and wall-clock time, at a matched seed budget.

  Read: Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS 2023

» **8 · Synthetic vs. human instruction data at matched size.** Does a model trained on model-generated instructions hold up on a held-out evaluation, and does a contamination check change the answer?

Read: Wang et al., "Self-Instruct," ACL 2023 · Zhou et al., "LIMA: Less Is More for Alignment," NeurIPS 2023

» **9 · Is DPO's implicit reward a good ranker?** Compare the implicit DPO reward against held-out preference labels and against an explicitly trained reward model, and characterize where they disagree.

Read: Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," NeurIPS 2023

» **10 · Does component X matter at small scale?** Ablate positional encoding, attention variant, or patch size in a ViT trained from scratch, and report variance across seeds so the effect is distinguishable from noise.

Read: Touvron et al., "Training data-efficient image transformers & distillation through attention," ICML 2021 · Steiner et al., "How to train your ViT?," TMLR 2022

» **11 · Distillation into a small model.** How much of the teacher–student gap closes on one narrow task, and — more interestingly — on which categories does it not close?

Read: Touvron et al., DeiT, ICML 2021 · Hinton, Vinyals & Dean, "Distilling the Knowledge in a Neural Network," NeurIPS 2014 Workshop

AREA C Multimodal Models

3 topics

» **12 · Build a probe set for a known CLIP weakness.** Counting, spatial relations, negation, or word order. Construct the probe set yourself, quantify the failure, and release it with the code.

Read: Yuksekgonul et al., "When and Why Vision-Language Models Behave like Bags-Of-Words," ICLR 2023 · Thrush et al., "Winoground," CVPR 2022 · Paiss et al., "Teaching CLIP to Count to Ten," ICCV 2023 · Zhai et al., "Sigmoid Loss for Language Image Pre-Training," ICCV 2023

» **13 · Fine-tune a small VLM and measure hallucination.** Chart QA, document QA, or a scientific-image task; define a hallucination metric and defend the definition.

Read: Liu et al., "Visual Instruction Tuning," NeurIPS 2023 · Li et al., "Evaluating Object Hallucination in Large Vision-Language Models," EMNLP 2023

» **14 · Does retrieval help on the long tail, or only on common concepts?** Split the evaluation set by concept frequency and report both halves separately.

Read: Ramos et al., "SmallCap: Lightweight Image Captioning Prompted with Retrieval Augmentation," CVPR 2023

AREA D Adversarial Attack & Defense

5 topics

» **15 · Transferability between CNN and ViT backbones.** Build a transfer matrix across four or more models and explain the asymmetries you find.

Read: Croce & Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," ICML 2020 · Naseer et al., "On Improving Adversarial Transferability of Vision Transformers," ICLR 2022

» **16 · Image-based prompt injection against a VLM.** How reliable is it, and what makes it fail? Use a controlled set of injected images and a small open model.

Read: Qi et al., "Visual Adversarial Examples Jailbreak Aligned Large Language Models," AAAI 2024

» **17 · How few poisoned samples are enough?** Backdoor a fine-tuned model and plot clean accuracy and attack success against the number of poisoned examples — then see whether a standard detector catches it.

Read: Carlini et al., "Poisoning Web-Scale Training Datasets is Practical," IEEE S&P 2024 · Wang et al., "Neural Cleanse," IEEE S&P 2019

» **18 · Diffusion-based purification, evaluated honestly.** What does purification cost in clean accuracy for what it buys in robust accuracy — under an **adaptive** attack, not just PGD?

Read: Nie et al., "Diffusion Models for Adversarial Purification," ICML 2022 · Lee & Kim, "Robust Evaluation of Diffusion-Based Adversarial Purification," ICCV 2023 · Athalye, Carlini & Wagner, "Obfuscated Gradients Give a False Sense of Security," ICML 2018

» **19 · Certified robustness: radius vs. cost.** Randomized smoothing on a small model. Be explicit about what the certificate does and does not guarantee.

Read: Cohen, Rosenfeld & Kolter, "Certified Adversarial Robustness via Randomized Smoothing," ICML 2019

AREA E Reproduce, Extend & Evaluate — 2 topics

» **20 · Reproduce and extend.** Take one CVPR / ICCV / ECCV / NeurIPS / ICLR paper from 2023–2026 with released code. Reproduce one table on a subset, then test a claim the authors did not — a new dataset, a removed component, a failure mode they never probed. The **extension** is the graded part; the reproduction is the entry ticket (§1.4).

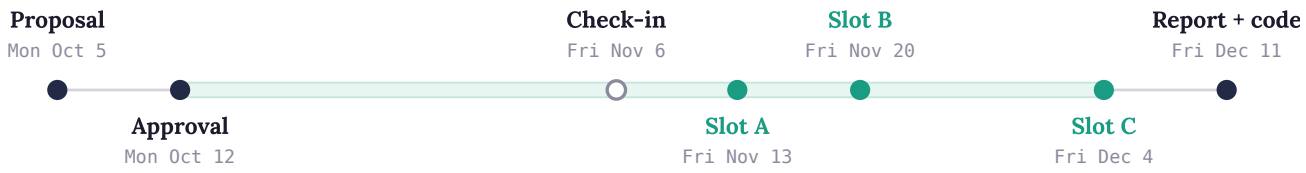
Read: the paper you pick, plus the two papers it is most often compared against

» **21 · Do our metrics measure what we assume?** Correlate FID and CLIPScore with a small human preference study run on your classmates — roughly 200 pairwise comparisons across two or three generators — and report rank correlation.

Read: Heusel et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," NeurIPS 2017 · Hessel et al., "CLIPScore," EMNLP 2021 · Jayasumana et al., "Rethinking FID: Towards a Better Evaluation Metric for Image Generation," CVPR 2024

PART 3 Timeline & Presentation Slots — 3 slots

3.1 Dates



The shaded stretch is your build window: approval on Oct 12 through your own presentation slot. The Nov 6 check-in is optional and ungraded — it exists so that a project which is quietly failing gets found in week four rather than on stage.

WHAT	WHERE	WHEN
Proposal (PDF)	Canvas	Mon, Oct 5 · 11:59 PM

WHAT	WHERE	WHEN
Revised proposal, if requested	Canvas	Mon, Oct 12 · 11:59 PM
Presentation slot sign-up	Shared sheet	Opens Oct 13, closes Oct 23
Progress check-in (optional)	Office hours	Week of Nov 2–6
Slides (PDF)	Canvas	11:59 PM the night before your slot
Presentation	In class	Nov 13, Nov 20, or Dec 4
Report (PDF) + repository link	Canvas	Fri, Dec 11 · 11:59 PM

You choose your slot; sign-up is first come, first served, with roughly a third of the class in each. Anyone who has not signed up by Oct 23 is assigned to Slot C. **The report and code deadline is Dec 11 for everyone**, whichever slot you presented in.

WHY PRESENT EARLY `Slot A ↔ easier bar`

Earlier slots are graded against a lower bar

There is no bonus-point adjustment. Instead, the *evidence you are expected to have in hand* scales with the extra weeks you had. Presenting on Nov 13 does not mean you stop working — it means the presentation is graded on where you were that day, and the report is graded on where you finished.

- **Slot A · Fri Nov 13 —** a working pipeline, your main result, and one baseline. Ablations may be *described as planned* rather than shown. Preliminary numbers are fine if labeled as preliminary. Slide design and rehearsal polish are not scored.
- **Slot B · Fri Nov 20 —** main result, baseline, and at least one completed ablation. One secondary experiment may be incomplete. Polish lightly weighted.
- **Slot C · Fri Dec 4 —** the full set: main result, baseline, ablations, and failure analysis, all complete. You have had three additional weeks. Fully weighted.

Everything else — the depth of the Q&A, the demand that you explain your own code, the fundamentals questions — is **identical in all three slots**. The leniency is about *how much you have finished*, never about *how well you understand it*.

PART 4

Grading

100 pts

4.0 How the 100 Points Are Split

COMPONENT	MEASURES	POINTS
Presentation, discussion & live Q&A	Whether you understand what you built, deeply enough to defend it under questioning without notes	65
Project report	Whether you can state the question, method, and evidence in writing, honestly	15
Code assessment	Whether the work is real, reproducible, and yours	15
Proposal	Whether you scoped a project that could succeed	5

Why the live component is 65 points. Writing and code can both be produced with a great deal of assistance, and in 2026 that assistance is extremely good. A report and a repository no longer tell me much on their own about what you know.

A live, unscripted conversation about your own work still does. When I ask why you chose that noise schedule, what your loss is actually penalizing, what happens to your defense under a different threat model, or why your validation curve has that shape at epoch 12 — the answer either exists or it does not.

The practical consequence: do not divide your effort in proportion to the artifacts. A polished report defending work you cannot explain will score poorly. Rough slides in front of work you know cold will score well.

4.1 Presentation, Discussion & Live Q&A

65 pts

10 minutes of presentation, then 6 minutes of open questions from the instructor and the class.

Slides are submitted the night before your slot and you present from the submitted file. No notes, no script, no reading from the slides. A live demo is welcome but never required, and a failed demo is not penalized if you can explain why it failed. *Timing may be adjusted once sign-up closes and the per-day count is known; any change is announced at least two weeks before Nov 13.*

CRITERION	FULL MARKS	PTS
Live Q&A and fundamentals	You answer questions about your own code, your design choices, and the underlying theory, immediately and correctly. When you do not know, you say so and reason out loud toward a defensible answer.	30
Technical exposition	The problem, the method, and the reason you chose that method are clear to someone who has not read your paper. You explain the mechanism, not just the pipeline diagram.	15
Results and evidence	The right comparison is shown, the baseline is fair, axes and metrics are defined, uncertainty is acknowledged, and the conclusion follows from the numbers on the slide.	12

CRITERION	FULL MARKS	PTS
Delivery and discussion	You finish on time, the slides are legible, and you engage seriously with questions — including as an audience member during your classmates' talks.	8

THE SEVEN KINDS OF QUESTION YOU WILL BE ASKED

Your code	"Open the file where the sampling loop lives and walk me through it."
Design choices	"Why this loss? What did you try before it, and why did you abandon that?"
Fundamentals	"What is the noise schedule actually doing?" · "What does the attention softmax normalize over, and why does that matter here?" · "Why does LoRA need so few parameters?"
Counterfactuals	"What happens to your result if I double the batch size?" · "Remove this term — predict the effect, then tell me whether you tested it."
Your data	"How many examples? Where did the validation split come from? How do you know there is no leakage?"
Your numbers	"How many seeds? What is the spread? Is that gap larger than your noise?"
Limits	"What is the strongest objection to this result, and what would you run next to address it?"

SCORING BANDS — Q&A PORTION (30 PTS)

Strong · 27–30	Answers immediate and correct, including on parts of the code you did not present. Handles counterfactuals by reasoning, not recall. Knows the limits of the work.
Solid · 22–26	Most questions answered correctly. Some hesitation on theory or on code written early. Recovers well when prompted.
Partial · 15–21	Can describe what the code does but not why. Fundamentals shaky. Counterfactuals produce guesses rather than reasoning.
Weak · 8–14	Cannot explain significant parts of the submitted work. Describes the pipeline at the level of the README.
Insufficient · 0–7	Cannot account for the submitted work at all.

The rule that matters most: if you cannot explain a section of your own code when asked, that section is treated as work you did not do, and the results depending on it are discounted across the code and report scores as well. This is not a penalty for using tools — it is the definition of what is being measured (§4.5).

4.2 Project Report — 15 pts

4–6 pages excluding references, two-column CVPR or NeurIPS style, PDF, due Fri Dec 11.

Appendices are permitted for extra figures and tables but are not read for grading unless the main text points to them.

Required sections: Abstract · Introduction and research question · Related work · Method · Experimental setup · Results · Ablations · Limitations and failure analysis · Conclusion · References · Contribution table (§1.4) · AI tool declaration (§4.5).

CRITERION	FULL MARKS	PTS
Problem & related work	The question is stated precisely in the first half page. Related work positions your project against specific papers, not a list of names.	3
Method	A reader in this class could reimplement the core of your method from the description. Notation defined; anything taken from existing code attributed.	3
Experiments & results	Setup reproducible from the text — seeds, splits, hyperparameters, hardware. Baselines fair. Tables and figures readable and captioned so they stand alone. Claims match the numbers.	5
Limitations & failure analysis	What did not work, why you think so, and what you would need to resolve it. Overclaiming costs points here; reporting a negative result clearly does not.	2
Writing, figures, citations	Clear prose, consistent notation, every figure referenced in the text, every borrowed idea and image cited.	2

Things that reliably lose points

- A results table with no baseline, or a baseline that was not tuned at all.
- Numbers in the text that disagree with numbers in the table.
- "We achieve significantly better performance" with one seed and no variance reported.
- Figures copied from the original paper without attribution, or terminal screenshots used as evidence.
- A limitations section that says only "more compute would help" — you had a cluster.

4.3 Code Assessment — 15 pts

Submit a repository link — GitHub, GitLab, or a zip if you cannot use either. Due Fri Dec 11.

Repository requirements

- `README.md` — what the project does, how to install, and **one command that reproduces your headline result**.
- `requirements.txt` or `environment.yml`, versions pinned.
- `CONTRIBUTIONS.md` — the table from §1.4, if you built on existing code.
- A **smoke test**: one command that runs end to end in under 10 minutes on a single GPU or a CPU, on a tiny subset, and produces *something*. This is the highest-value thing in your repository, because it is what gets run first.
- Your GAIVI or Slurm job scripts, if you used the cluster — they are part of how the result was produced.
- Seed control — a seed that is set, logged, and configurable.

- Data handling — a download script or precise instructions. Do not commit datasets or checkpoints over 100 MB; link them.
- Real commit history. Your work should appear as commits over weeks, not one upload on Dec 11.

CRITERION	FULL MARKS	PTS
Reproducibility	A clean clone installs and the smoke test runs without me editing anything. The headline command produces numbers consistent with the report.	5
Evaluation correctness	The metric code does what the report says. No train/test leakage, no evaluating on the tuning split, no silently different preprocessing between your method and the baseline.	4
Provenance	Base commit pinned, `CONTRIBUTIONS.md` accurate, commit history showing the work develop over time.	3
Organization & clarity	Sensible structure, no dead code or unused notebooks in the root, configuration separated from logic, names that say what they hold.	3

What I will actually do: clone it fresh, follow the README exactly, run the smoke test, then read the evaluation code closely. Evaluation bugs are the most common and most costly problem in submitted projects — a leaked split can make a mediocre method look excellent, and those numbers will not survive the Q&A. Code quality is graded generously; correctness is not. **Messy but correct beats tidy but wrong.**

4.4 Undergraduate (4930) vs. Graduate (6930)

The scale, the weights, and the deliverables are identical. What differs is the bar each component is measured against, and how deep the Q&A goes.

Your level is fixed by the enrollment you declare in the proposal header (§1.5) and cannot be changed afterward.

COMPONENT	UNDERGRADUATE — 4930	GRADUATE — 6930
Contribution	Reproduce a method and extend it in one clearly defined way — a new dataset, a new ablation, a new evaluation. The extension may be modest as long as it is real and you designed it.	A contribution that would be a defensible paragraph in a workshop paper: a new component, a new attack or defense, a corrective result, or an evaluation that changes how the original claim should be read.
Experiments	One main comparison against one baseline, plus one ablation. Single-seed results acceptable if labeled.	Multiple baselines where they exist, at least two ablations, results over ≥ 3 seeds with variance reported, and a stated reason for any comparison you chose not to run.
Q&A depth	Explain your pipeline, your code, your design choices, and the standard mechanisms you used at the level covered in lecture.	All of that, plus the mathematics behind the method, its assumptions, when it fails, how it relates to the alternatives you did not pick, and what the next experiment would be.

COMPONENT	UNDERGRADUATE — 4930	GRADUATE — 6930
Related work	5–8 references, correctly positioned.	12–20 references, including the competing approaches, with an argument about where your work sits among them.
Report	Clear, complete, honest. Writing graded on clarity.	Publication-adjacent: precise notation, a methods section someone could implement from, and a limitations section that anticipates a reviewer's objections.
Code	Reproducible and correct.	Reproducible, correct, and structured so someone else could extend it — configuration separated, experiments scriptable, results regenerable.

What this looks like in the Q&A. Both levels get asked "why this loss?" An undergraduate is expected to explain what the loss penalizes and why that suits the task. A graduate student is expected to do that, *and* to say what the gradient does near the failure case, what the loss assumes about the data distribution, and what the two obvious alternatives would have traded away.

The undergraduate bar is lower. It is not low.

4.5 Using AI Coding Assistants

This is a generative AI course. Telling you not to use generative AI tools would be strange, and unenforceable besides. So the policy is simple.

Permitted, without restriction

- Writing, debugging, refactoring, and explaining code.
- Understanding papers, deriving equations, exploring alternatives.
- Drafting, editing, and improving your writing.
- Generating figures, tests, and scaffolding.

Not permitted

- Any AI assistance **during the live Q&A**. No devices, no earpieces, no second screen.
- Submitting results you did not produce — fabricated numbers, invented citations, plots not generated by your code.
- Failing to declare the tools you used.

REQUIRED DECLARATION — PROPOSAL AND REPORT

```
AI tool declaration: I used a coding assistant to implement the data loader
and to debug the sampling loop, and to edit the introduction and related work
for clarity. The experimental design, the ablation choices, the evaluation
code, and all reported numbers are mine.
```

How understanding is verified. Not by detectors, not by style analysis, not by inspecting your commit timestamps — by the 30-point live Q&A in §4.1. If a tool wrote a function and you understood what it wrote, you will answer the question about it and lose nothing. If a tool wrote a function and you did not read it, you will not, and that section is treated as work you did not do.

That is the whole design of this grading scheme. The tools are excellent and getting better; the thing worth grading is what is in your head when the tool is switched off.

Submission Checklist

- [ ] **Proposal (Oct 5):** two pages, PDF, named `lastname_proposal.pdf`. Header states 4930 or 6930. Primary paper named. Falsifiable outcome stated. Compute plan with single-GPU fallback. Contribution table if building on existing code.
- [ ] **Slides (night before your slot):** PDF on Canvas. You present from the submitted file.
- [ ] **Report (Dec 11):** 4–6 pages, two-column CVPR/NeurIPS style, PDF, named `lastname_report.pdf`. Every number in it produced by the code you submitted.
- [ ] **Repository (Dec 11):** link on Canvas. Clean clone installs; smoke test finishes in under 10 minutes; `CONTRIBUTIONS.md` present and accurate; base commit pinned; seeds set and logged.
- [ ] **Reproducibility:** record the GPU used for every reported result. Results in the writeup must match re-running the code end to end.
- [ ] **Before you walk in:** you have opened every file in your repository and can explain what it does, and you can answer the seven kinds of question in §4.1 out loud.

COMPONENT	POINTS
Proposal	5
Presentation, discussion & live Q&A	65
Project report	15
Code assessment	15
Total	**100**

Late work

Proposal	−1 point per day, up to 3 days; not accepted after Oct 8
Slides	−2 points off the presentation score if not submitted before your slot
Report and code	−10% of that component per day, up to 5 days

Presentation	Cannot be late. Missing your slot scores 0 unless you have a documented emergency, in which case you present in the next available slot and are graded against **that** slot's bar (§3.1).

Academic integrity

The following are violations and are referred, not negotiated: submitting code, text, figures, or results from another source without declaring it — including another student's work, a public repository, or an AI tool; reporting numbers your code did not produce; citing papers you did not read or that do not exist; presenting work you did not do.

Talking to classmates about methods, debugging together, and sharing pointers to papers and datasets are all encouraged. Sharing code, results, or report text is not.